

Assignment 2-Case Study

Stage 3 Tutorial Activities

From Requirements to Domain Models

Week 6 | 60 minutes

Candidate Concepts

Candidate	Class?	Reason
Patient	YES	Represents a primary real-world person in the system with distinct identity, state, and behavior.
Practitioner	YES	Represents a healthcare provider in the system with distinct identity, attributes, and responsibilities
Appointment	YES	Connects a Patient to a Practitioner at a specific time, maintaining its own complex lifecycle states and business rules.
Name	NO	It is a simple data value describing a property of a Patient or Practitioner, not an independent entity with behaviors.
Clinic	NO	Represents the overarching environment in which the system operates. Unless managing a multi-clinic chain, treating it as a class introduces unnecessary overhead.
Database	NO	It is an implementation detail for storing data, not a domain concept within the business problem logic.
Cancellation	NO	Cancellation is an action that modifies an appointment's state or a state value (AppointmentStatus.CANCELLED) rather than a standalone domain object.
Status	NO	Represents the current state of an Appointment object (SCHEDULED, ARRIVED, IN_PROGRESS, etc.) rather than an independent class.

CRC Cards

Patient

Responsibilities	Collaborators
Maintain patient ID, contact information, and personal details.	Appointment (stores references to historical and active booking objects)
Maintain a historical log of all past and present appointments linked to the patient.	Receptionist (invokes registration and contact updates)

Practitioner

Responsibilities	Collaborators
Maintain practitioner profile details, medical specialty, and allocated consultation room number.	Appointment (inspects assigned booking dates, times, and active statuses)
Filter and return assigned daily appointment schedules for a requested date.	Receptionist (queries practitioner availability and schedules)

Appointment

Responsibilities	Collaborators
Enforce booking collision checks for a practitioner and control valid lifecycle status transitions.	Patient (validates patient reference and updates patient booking history).
Enforce cancellation rules by mandating a non-empty cancellation reason text when moving to the cancelled state.	Practitioner (validates practitioner reference and checks room/time schedule availability).

Relationship Reasoning

Patient to Appointment: which relationship and why?

- Relationship: Association (specifically, 1-to-Many Association).
- Patient and an Appointment are independent real-life entities with distinct lifespans.
- A Patient is associated with zero, one, or many Appointment objects over time.
- If an Appointment is deleted or cancelled, the Patient entity continues to exist in the system.
- Therefore, it is a standard directional/bidirectional association rather than composition or inheritance.

Practitioner to Appointment: what multiplicity?

- Multiplicity: 1 to 0..* (One Practitioner to Zero-or-Many Appointments).
- A single Practitioner can be assigned to multiple appointments over time (0..*), but only one active appointment at any specific time slot.
- Every individual Appointment MUST be assigned to exactly one (1).

Should Appointment inherit from Patient?

Answer: No.

Why: Inheritance represents an "IS-A" relationship. An Appointment is not a type of Patient.

Does Clinic need to own every object?

Answer: No

Why: Clinic is best modeled as a top-level Container / Registry / System Controller that holds collections (lists or dictionaries) of Patient, Practitioner, and Appointment objects to coordinate lookups.

AI Model Critique

Critique AI proposals: PatientManager, PractitionerManager, AppointmentManager, ClinicController, NotificationManager, ScheduleEngine.

CRITIQUE:

SmartCare requirements describe a domain model centered on:

Patient

Practitioner

Appointment

The solution statement specifically identifies these three classes as the core of the system. Creating a separate "Manager" for every domain entity strips behavior out of core objects, leading to a Domain Model where entities become passive data holders and managers become bloated procedural scripts.

If an application has menus or acts as a coordinator, a controller like ClinicController class can make sense.

The controller could coordinate:

- registering patients
- registering practitioners
- creating appointments
- generating reports

This class would be implementation infrastructure rather than a business entity.